ЛЕКЦИЯ 7. FLUTTER: ПРОДВИНУТЫЕ ТЕМЫ

АНИМАЦИИ В FLUTTER

ТИПЫ АНИМАЦИЙ

НЕЯВНЫЕ (IMPLICIT)

- AnimatedContainer
- AnimatedOpacity
- AnimatedPositioned
- AnimatedAlign
- AnimatedDefaultTextStyle

ЯВНЫЕ (EXPLICIT)

- AnimationController
- Tween
- CurvedAnimation
- Hero
- CustomPainter

НЕЯВНЫЕ АНИМАЦИИ

```
class AnimatedBox extends StatefulWidget {
  @override
  State<AnimatedBox> createState() => _AnimatedBoxState();
}

class _AnimatedBoxState extends State<AnimatedBox> {
  bool _expanded = false;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () => setState(() => _expanded = !_expanded),
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 500),
        curve: Curves.easeInOut,
        width: _expanded ? 200 : 100,
        height: _expanded ? 200 : 100,
        color: _expanded ? Colors.blue : Colors.red,
        alignment: Alignment.center,
        child: const Text('Нажми!', style: TextStyle(color: Colors.
```

```
        ),
      );
    }
}
```

ЯВНЫЕ АНИМАЦИИ

```
class RotateWidget extends StatefulWidget {
  @override
  State<RotateWidget> createState() => _RotateWidgetState();
}

class _RotateWidgetState extends State<RotateWidget>
    with SingleTickerProviderStateMixin {

  late AnimationController _controller;
  late Animation<double> _animation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: const Duration(seconds: 2),
      vsync: this,
    );
    _animation = Tween<double>(begin: 0, end: 1).animate(
      CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
    );
    _controller.repeat(reverse: true);
```

```
  }

  @override
  Widget build(BuildContext context) {
    return RotationTransition(
      turns: _animation,
      child: const Icon(Icons.refresh, size: 100),
    );
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
}
```

HERO-АНИМАЦИЯ

```
// Экран 1
Hero(
  tag: 'avatar',
  child: CircleAvatar(
    backgroundImage: NetworkImage('https://example.com/photo.j
  ),
)

// Экран 2
Hero(
  tag: 'avatar',  // тот же tag!
  child: Image.network('https://example.com/photo.jpg'),
)

// Navigator с поддержкой Hero
Navigator.push(
  context,
  PageRouteBuilder(
    pageBuilder: (_, __, ___) => DetailScreen(),
```

```
    transitionsBuilder: (_, animation, __, child) {
      return FadeTransition(opacity: animation, child: child);
    },
  ),
);
```

ТЕСТИРОВАНИЕ

ВИДЫ ТЕСТИРОВАНИЯ

UNIT TESTS

Тестирование логики, функций, классов

WIDGET TESTS

Тестирование виджетов в изоляции

INTEGRATION TESTS

Тестирование полного сценария

UNIT TESTS

```
// test/counter_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/counter.dart';

void main() {
  group('Counter', () {
    test('initial value is 0', () {
      final counter = Counter();
      expect(counter.value, 0);
    });

    test('increment increases value', () {
      final counter = Counter();
      counter.increment();
      expect(counter.value, 1);
    });

    test('decrement decreases value', () {
      final counter = Counter();
      counter.decrement();
```

```
      expect(counter.value, -1);
    });
  });
}

// Запуск: flutter test
```

WIDGET TESTS

```
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/main.dart';

void main() {
  testWidgets('Counter increments test', (WidgetTester tester) asyn
    await tester.pumpWidget(const MyApp());

    // Проверяем начальное значение
    expect(find.text('0'), findsOneWidget);
    expect(find.text('1'), findsNothing);

    // Нажимаем кнопку
    await tester.tap(find.byIcon(Icons.add));
    await tester.pump();

    // Проверяем результат
    expect(find.text('0'), findsNothing);
    expect(find.text('1'), findsOneWidget);
  });
}
```

```
// Запуск: flutter test
```

CI/CD

CI/CD ДЛЯ FLUTTER

ИНСТРУМЕНТЫ

- **GitHub Actions**
- **CodeMagic** (специализированный)
- **Bitrise**
- **GitLab CI/CD**

ЭТАПЫ

1. Анализ кода (dart analyze)
2. Тесты (flutter test)
3. Сборка (flutter build)
4. Деплой в App Store / Google Play

GITHUB ACTIONS: ПРИМЕР

```yaml
name: Flutter CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.24.x'
          channel: 'stable'

      - name: Install dependencies
        run: flutter pub get

      - name: Analyze
        run: flutter analyze

      - name: Test
        run: flutter test
```

```
    - name: Build APK
      run: flutter build apk --release
```

FLUTTER WEB И DESKTOP

FLUTTER WEB

```
# Запуск в браузере
flutter run -d chrome

# Сборка для веба
flutter build web --release

# Результат в build/web/
# Можно хостить на любом веб-сервере
```

- Кроссплатформенный: тот же код для Web
- Рендеринг: HTML или CanvasKit
- SEO: ограниченный (SPA)
- Производительность: хуже нативного веба, но лучше PWA

FLUTTER DESKTOP

```
# Сборка для десктопа
flutter build linux    # Linux
flutter build windows  # Windows
flutter build macos    # macOS

# Запуск
flutter run -d windows
flutter run -d linux
flutter run -d macos
```

- Тот же код для мобильных и десктопных платформ
- Оконный интерфейс, меню, клавиатура
- С 2024: стабильная поддержка

ПРОИЗВОДИТЕЛЬНОСТЬ

DEVTOOLS: ПРОФИЛИРОВАНИЕ

- **Flutter DevTools** - набор инструментов для анализа
- Запуск: `flutter run --profile`
- Инструменты:
  - **Widget Inspector** - дерево виджетов
  - **Timeline View** - производительность
  - **Memory View** - утечки памяти
  - **Network View** - сетевые запросы

ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ

```
// 1. Используйте const конструкторы
const Text('Hello')  // лучше
Text('Hello')        // хуже

// 2. Избегайте重建 тяжёлых виджетов
class ParentWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        const HeavyWidget(),  // не пересоздаётся
        Text('Другой виджет'),
      ],
    );
  }
}
```

```
// 3. Используйте ListView.builder для длинных списков
ListView.builder(itemCount: 10000, ...)  // лучше
ListView(children: List.generate(10000, ...))  // хуже

// 4. Кэшируйте изображения
CachedNetworkImage(imageUrl: '...')  // лучше
Image.network('...')  // без кэша
```

ЛОКАЛИЗАЦИЯ

FLUTTER: ЛОКАЛИЗАЦИЯ

```
# l10n/app_en.arb
{
  "appTitle": "My App",
  "hello": "Hello, {name}!",
  "@hello": {
    "placeholders": {
      "name": {"type": "String"}
    }
  }
}

# l10n/app_ru.arb
{
  "appTitle": "Моё приложение",
  "hello": "Привет, {name}!"
}

# Использование в коде
Text(AppLocalizations.of(context)!.hello('Иван'))
```

ОКРУЖЕНИЯ (ENVIRONMENTS)

FLUTTER: ОКРУЖЕНИЯ

```
// pubspec.yaml
flutter:
  assets:
    - .env.development
    - .env.staging
    - .env.production

// lib/env.dart
class Env {
  static const String apiUrl = String.fromEnvironment(
    'API_URL',
    defaultValue: 'https://api.example.com',
  );
}

// Запуск с окружением
flutter run --dart-define=API_URL=https://dev.api.example.com
flutter run --dart-define=API_URL=https://staging.api.example.com

// Сборка
```

```
flutter build apk --dart-define=API_URL=https://prod.api.example.co
```

ПАКЕТЫ И ПЛАГИНЫ

ПОПУЛЯРНЫЕ ПАКЕТЫ FLUTTER (2025)

Пакет	Назначение
provider / riverpod	Управление состоянием
go_router	Декларативная навигация
dio	HTTP-клиент
hive	Быстрое локальное хранилище
sqflite	SQLite
shared_preferences	Простые настройки
cached_network_image	Кэширование изображений
flutter_local_notifications	Локальные уведомления
image_picker	Выбор фото/камера
permission_handler	Разрешения
firebase_core	Firebase SDK
flutter_bloc	BLoC-паттерн

ANIMATEDBUILDER: ПЕРЕИСПОЛЬЗОВАНИЕ

```
class PulseWidget extends StatefulWidget {
  @override
  State<PulseWidget> createState() => _PulseWidgetState();
}

class _PulseWidgetState extends State<PulseWidget>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: const Duration(milliseconds: 1200),
      vsync: this,
    )..repeat(reverse: true);
  }
```

```
  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: _controller,
      builder: (context, child) {
        return Transform.scale(
          scale: 1.0 + (_controller.value * 0.2),
          child: child,
        );
      },
      child: const Icon(Icons.favorite, size: 80, color: Colors.red
    );
  }
}
```

STAGGERED ANIMATIONS

```
// Последовательные анимации
class StaggeredDemo extends StatefulWidget {
  @override
  State<StaggeredDemo> createState() => _StaggeredDemoState();
}

class _StaggeredDemoState extends State<StaggeredDemo>
    with TickerProviderStateMixin {
  late AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: const Duration(milliseconds: 2000),
      vsync: this,
    );
  }

  @override
  Widget build(BuildContext context) {
    final fade = Tween<double>(begin: 0, end: 1).animate(
```

```
      CurvedAnimation(parent: _controller, curve: Interval(0.0, 0.5)),
    );
    final slide = Tween<Offset>(
      begin: const Offset(0, 0.5),
      end: Offset.zero,
    ).animate(
      CurvedAnimation(parent: _controller, curve: Interval(0.3, 0.8)),
    );
    // Запуск
    _controller.forward();
    return FadeTransition(
      opacity: fade,
      child: SlideTransition(position: slide, child: Text('Hello!')),
    );
  }
}
```

RIVE И LOTTIE: АНИМАЦИИ ИЗ ФАЙЛОВ

RIVE

- Интерактивные векторные анимации
- Создание в Rive Editor
- State Machines для логики
- pub: rive

LOTTIE

- Анимации из After Effects
- Формат JSON
- Лёгкие и масштабируемые
- pub: lottie

```
// Lottie
Lottie.asset('assets/animation.json',
  width: 200, height: 200, repeat: true);

// Rive
RiveAnimation.asset('assets.riv',
  artboard: 'Main', stateMachineName: 'StateMachine');
```

PAGE TRANSITIONS

```
// Кастомный переход между экранами
class FadePageRoute extends PageRouteBuilder {
  final Widget page;
  FadePageRoute({required this.page})
      : super(
          pageBuilder: (_, __, ___) => page,
          transitionsBuilder: (_, animation, __, child) {
            return FadeTransition(opacity: animation, child: child);
          },
          transitionDuration: const Duration(milliseconds: 400),
        );
}

// Использование
Navigator.push(context, FadePageRoute(page: DetailScreen()));

// Или через GoRouter
GoRoute(
  path: '/detail',
  pageBuilder: (context, state) => CustomTransitionPage(
```

```
    child: DetailScreen(),
    transitionsBuilder: (context, animation, secondaryAnimation, ch
      return SlideTransition(
        position: Tween<Offset>(
          begin: const Offset(1, 0),
          end: Offset.zero,
        ).animate(animation),
        child: child,
      );
    },
  ),
)
```

INTEGRATION TESTS

```
// integration_test/app_test.dart
import 'package:integration_test/integration_test.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('Full login flow', (tester) async {
    app.main();
    await tester.pumpAndSettle();

    // Вводим email
    await tester.enterText(find.byKey(Key('email')), 'test@mail.com
    // Вводим пароль
    await tester.enterText(find.byKey(Key('password')), '123456');
    // Нажимаем кнопку входа
    await tester.tap(find.byKey(Key('login_button')));
    await tester.pumpAndSettle();
```

```
    // Проверяем, что перешли на главный экран
    expect(find.text('Добро пожаловать!'), findsOneWidget);
  });
}

// Запуск: flutter test integration_test/
```

GOLDEN TESTS: СРАВНЕНИЕ СКРИНШОТОВ

```
// Тестирование визуального вида виджета
testWidgets('Button looks correct', (tester) async {
  await tester.pumpWidget(MaterialApp(
    home: Scaffold(
      body: ElevatedButton(
        onPressed: () {},
        child: Text('Нажми'),
      ),
    ),
  ));

  // Сравнение с "золотым" скриншотом
  await expectLater(
    find.byType(ElevatedButton),
    matchesGoldenFile('goldens/button.png'),
  );
});
```

```
// Обновление золотых файлов:
// flutter test --update-goldens
```

MOCKING С MOCKITO

```
import 'package:mockito/annotations.dart';
import 'package:mockito/mockito.dart';

@GenerateMocks([UserRepository, ApiService])
void main() {
  test('fetchUser returns user', () async {
    final mockRepo = MockUserRepository();
    when(mockRepo.getUser(1)).thenAnswer(
      (_) async => User(id: 1, name: 'Тест'),
    );

    final user = await mockRepo.getUser(1);
    expect(user.name, 'Тест');
    verify(mockRepo.getUser(1)).called(1);
  });
}

// Генерация моков:
// dart run build_runner build --delete-conflicting-outputs
```

BUILD_RUNNER И КОДОГЕНЕРАЦИЯ

```
# Установка
dart pub add --dev build_runner
dart pub add --dev json_serializable
dart pub add --dev freezed_annotation
dart pub add freezed_annotation

# Генерация
dart run build_runner build          # одноразово
dart run build_runner watch          # автоматически

# pubspec.yaml
dependencies:
  freezed_annotation: ^2.4.0
  json_annotation: ^4.8.0
dev_dependencies:
  build_runner: ^2.4.0
  freezed: ^2.5.0
  json_serializable: ^6.7.0
```

FREEZED: IMMUTABLE МОДЕЛИ

```dart
import 'package:freezed_annotation/freezed_annotation.dart';
import 'package:flutter/foundation.dart';

part 'user.freezed.dart';
part 'user.g.dart';

@freezed
class User with _$User {
  const factory User({
    required int id,
    required String name,
    required String email,
    @Default(false) bool isVerified,
  }) = _User;

  factory User.fromJson(Map<String, dynamic> json) =>
      _$UserFromJson(json);
}

// Автоматически генерируются:
// - copyWith()
// - ==
```

```
// - hashCode
// - toString()
// - toJson/fromJson

final user1 = User(id: 1, name: 'Иван', email: 'ivan@mail.ru');
final user2 = user1.copyWith(name: 'Пётр');
```

АРХИТЕКТУРНЫЕ ПАТТЕРНЫ

MVVM

- **Model** - данные
- **View** - UI
- **ViewModel** - логика
- Популярен в Flutter

CLEAN ARCHITECTURE

- **Presentation** - UI + ViewModel
- **Domain** - бизнес-логика
- **Data** - репозитории, API
- Максимальная тестируемость

СТРУКТУРА ПРОЕКТА (FEATURE-FIRST)

```
lib/
├── core/
│   ├── di/                 # Dependency Injection
│   ├── network/            # API клиент, интерцепторы
│   ├── theme/              # Темы, цвета, стили
│   └── utils/              # Утилиты
├── features/
│   ├── auth/
│   │   ├── data/           # UserRepository, AuthApi
│   │   ├── domain/         # User model, UseCases
│   │   └── presentation/# LoginScreen, AuthBloc
│   ├── home/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   └── settings/
├── l10n/                   # Локализация
├── main.dart
```

FIREBASE: ИНТЕГРАЦИЯ

```
# pubspec.yaml
dependencies:
  firebase_core: ^3.8.0
  firebase_auth: ^5.4.0
  cloud_firestore: ^5.6.0
  firebase_messaging: ^15.2.0
  firebase_analytics: ^11.4.0

# Инициализация
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}

// Firestore
final users = FirebaseFirestore.instance.collection('users');
await users.add({'name': 'Иван', 'email': 'ivan@mail.ru'});
final snapshot = await users.get();
```

FIREBASE AUTHENTICATION

```
// Email/Password
await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: 'user@mail.com',
  password: '123456',
);

// Google Sign-In
final googleUser = await GoogleSignIn().signIn();
final googleAuth = await googleUser!.authentication;
final credential = GoogleAuthProvider.credential(
  accessToken: googleAuth.accessToken,
  idToken: googleAuth.idToken,
);
await FirebaseAuth.instance.signInWithCredential(credential);

// Слушатель состояния
FirebaseAuth.instance.authStateChanges().listen((User? user) {
  if (user != null) {
    print('Вошёл: ${user.email}');
  } else {
```

```
    print('Вышел');
  }
});
```

ДОСТУПНОСТЬ (ACCESSIBILITY)

```
// Semantics для screen readers
Semantics(
  label: 'Кнопка входа',
  hint: 'Нажмите для входа в приложение',
  button: true,
  child: ElevatedButton(
    onPressed: () {},
    child: Text('Войти'),
  ),
)

// Проверка доступности
// Flutter DevTools -> Accessibility Inspector

// Рекомендации:
// - Размер.touch targets >= 48x48 dp
// - Контраст текста >= 4.5:1
// - Не используйте цвет как единственное средство информации
```

```
// - Тестируйте с TalkBack (Android) / VoiceOver (iOS)
```

ЛОГИРОВАНИЕ И МОНИТОРИНГ

```
// Логирование
import 'package:logging/logging.dart';

final _log = Logger('MyApp');

void main() {
  Logger.root.level = Level.ALL;
  Logger.root.onRecord.listen((record) {
    print('${record.level.name}: ${record.time}: ${record.message}'
  });
}

_log.info('Загрузка данных...');
_log.warning('Медленный запрос: ${elapsed}ms');
_log.severe('Ошибка сети: $error');

// Sentry для мониторинга ошибок
import 'package:sentry_flutter/sentry_flutter.dart';

await SentryFlutter.init(
```

```
  (options) {
    options.dsn = 'https://...@sentry.io/...';
    options.tracesSampleRate = 1.0;
  },
  appRunner: () => runApp(MyApp()),
);
```

DEPENDENCY INJECTION: GET_IT

```
import 'package:get_it/get_it.dart';

final getIt = GetIt.instance;

void setup() {
  // Singleton (один экземпляр)
  getIt.registerSingleton<ApiService>(ApiService());

  // Lazy singleton (создаётся при первом обращении)
  getIt.registerLazySingleton<UserRepository>(
    () => UserRepository(getIt<ApiService>()),
  );

  // Factory (создаётся каждый раз)
  getIt.registerFactory<UserBloc>(
    () => UserBloc(getIt<UserRepository>()),
  );
}

// Использование
```

```
final api = getIt<ApiService>();
final repo = getIt<UserRepository>();
```

КАЧЕСТВО КОДА

```
# analysis_options.yaml
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    - prefer_const_constructors
    - prefer_const_declarations
    - avoid_print
    - prefer_single_quotes
    - require_trailing_commas
    - prefer_final_locals
    - avoid_unnecessary_containers
    - sized_box_for_whitespace

analyzer:
  errors:
    missing_return: error
    dead_code: warning
  exclude:
    - "**/*.g.dart"
```

```
  - "**/*.freezed.dart"
```

ПУБЛИКАЦИЯ ПАКЕТОВ

```
# pub.dev - центральный репозиторий пакетов

# Проверка пакета перед публикацией
dart pub publish --dry-run

# Публикация
dart pub publish

# Требования:
# - Описание и README
# - Лицензия (LICENSE)
# - pubspec.yaml с версией
# - dart doc (документация)
# - Оценка пакета (популярность, поддержка)

# Melos для monorepo
# pub.dev/packages/melos
melos bootstrap
melos run analyze
```

```
melos run test
```

FIREBASE CLOUD MESSAGING

```
// pub.dev/packages/firebase_messaging
import 'package:firebase_messaging/firebase_messaging.dart';

// Обработка push-уведомлений в foreground
FirebaseMessaging.onMessage.listen((RemoteMessage message) {
  print('Получено: ${message.notification?.title}');
  showCustomNotification(message);
});

// Обработка при открытии приложения через уведомление
FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message)
  navigateToScreen(message.data['screen']);
});

// Получение токена
final token = await FirebaseMessaging.instance.getToken();
print('FCM Token: $token');
```

РАЗРАБОТКА ПЛАГИНОВ

```
# Создание плагина
flutter create --template=plugin -a kotlin -i swift my_plugin

# Структура
my_plugin/
├── lib/my_plugin.dart          # Dart API
├── android/
│   └── src/main/kotlin/...     # Android реализация
├── ios/Classes/                 # iOS реализация
├── example/                    # Пример использования
└── pubspec.yaml

# pubspec.yaml плагина
flutter:
  plugin:
    platforms:
      android:
        package: com.example.my_plugin
        pluginClass: MyPlugin
      ios:
```

```
pluginClass: MyPlugin
```

DART FFI: ВЫЗОВ C/C++ КОДА

```
import 'dart:ffi';
import 'package:ffi/ffi.dart';

// Определение функции
typedef NativeAdd = Int32 Function(Int32 a, Int32 b);
typedef DartAdd = int Function(int a, int b);

// Загрузка библиотеки
final dylib = DynamicLibrary.open('libmath.so');
final add = dylib.lookupFunction<NativeAdd, DartAdd>('add');

// Использование
final result = add(2, 3);  // 5

// Применение:
// - Вычисления на нативной стороне (Benchmark)
// - Работа с нативными API
// - Интеграция с существующими C/C++ библиотеками
```

IMPELLER: НОВЫЙ ДВИЖОК РЕНДЕРИНГА

- Замена Skia начиная с Flutter 3.16 (2024)
- **Предкомпиляция шейдеров** - нет джиттера при первом рендеринге
- Более предсказуемая производительность
- Меньшее энергопотребление
- Оптимизирован для Vulkan (Android) и Metal (iOS)

```
# Включение Impeller
flutter run --enable-impeller

# Для Android: в AndroidManifest.xml
```

```
<application android:enableOnBackInvokedCallback="true"
    ... >
    <meta-data
        android:name="io.flutter.embedding.android.EnableI
        android:value="true" />
</application>
```

DEVTOOLS: WIDGETS INSPECTOR

- Просмотр дерева виджетов в реальном времени
- Измерение размеров и отступов
- Поиск "лишних" rebuild'ов
- Свойства виджетов и их значения
- Сравнение двух состояний (diff)

```
# Запуск DevTools
flutter run
# DevTools откроется автоматически

# Или вручную
dart devtools
```

DEVTOOLS: TIMELINE VIEW

- Визуализация UI и GPU кадров
- Поиск "ѕшивающих" кадров (>16ms)
- Анализ Build, Layout, Paint фаз
- Идентификация тяжёлых виджетов

```
// Включение рамок производительности
MaterialApp(
  debugShowCheckedModeBanner: false,
  showPerformanceOverlay: true,  // показывает overlay
)

// Или через DevTools
// Performance -> Timeline -> Record
```

РЕЖИМЫ СБОРКИ FLUTTER

Режим	Компиляция	Debug	Производительность	Использование
debug	JIT	Да	Низкая	Разработка
profile	AOT	Ограниченно	Близка к release	Профилирование
release	AOT	Нет	Максимальная	Production

```
flutter run --debug          # разработка
flutter run --profile        # профилирование
flutter run --release        # production
flutter build apk --release # сборка APK
```

WIDGET REBUILD OPTIMIZATION

```
// Плохо: пересоздаётся каждый раз
class BadWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('${DateTime.now()}');  // пересоздаётся!
  }
}

// Хорошо: const конструктор
class GoodWidget extends StatelessWidget {
  const GoodWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Статический текст');
  }
}

// Хорошо: выносим изменяемую часть
class OptimizedWidget extends StatelessWidget {
```

```
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      const Text('Заголовок'),       // const
      Text('${DateTime.now()}'),     // только это меняется
    ]);
  }
}
```

FLUTTER ROADMAP: ЧТО ИЗУЧАТЬ ДАЛЬШЕ

ПРОДВИНУТОЕ

- Isolates и Compute
- FFI и нативные вызовы
- Разработка плагинов
- Impeller и рендеринг

ЭКОСИСТЕМА

- Firebase интеграция
- CI/CD pipelines
- Мониторинг (Sentry)
- Публикация пакетов

FLUTTER: ТРЕНДЫ 2025-2026

- **Impeller** - стандартный рендеринг на всех платформах
- **dart compile** - AOT компиляция для серверов
- **Flutter GPU** - низкоуровневая графика
- **Unified platform** - один код для mobile/web/desktop/embedded
- **AI-assisted development** - генерация кода, UI из промптов
- **WebAssembly (Wasm)** - Flutter в браузере через Wasm

ИТОГИ КУРСА

НАТИВНАЯ (3 ЛЕКЦИИ)

- Java, Kotlin, JVM
- Экосистема Android
- Компоненты приложений

КРОССПЛАТФОРМА (1 ЛЕКЦИЯ)

- PWA, Cordova
- React Native
- Flutter (обзор)

FLUTTER (3 ЛЕКЦИИ)

- Дарт, виджеты
- Навигация, состояние
- Анимации, тестирование

ЧТО ДАЛЬШЕ?

- **Практика** - 5 лабораторных работ
- **Проект** - создайте приложение на выбор
- Изучайте документацию: flutter.dev, developer.android.com
- Следите за трендами: Compose, Flutter 4.x, AI в мобильной разработке